How-To: Create your first Process

> ⚠ **Warning**
>
> The *Semantic Kernel Process Framework* is experimental, still in development and is subject to change.

Overview

The Semantic Kernel Process Framework is a powerful orchestration SDK designed to simplify the development and execution of AI-integrated processes. Whether you are managing simple workflows or complex systems, this framework allows you to define a series of steps that can be executed in a structured manner, enhancing your application's capabilities with ease and flexibility.

Built for extensibility, the Process Framework supports diverse operational patterns such as sequential execution, parallel processing, fan-in and fan-out configurations, and even map-reduce strategies. This adaptability makes it suitable for a variety of real-world applications, particularly those that require intelligent decision-making and multi-step workflows.

Getting Started

The Semantic Kernel Process Framework can be used to infuse AI into just about any business process you can think of. As an illustrative example to get started, let's look at building a process to generate documentation for a new product.

Before we get started, make sure you have the required Semantic Kernel packages installed:

.NET CLI

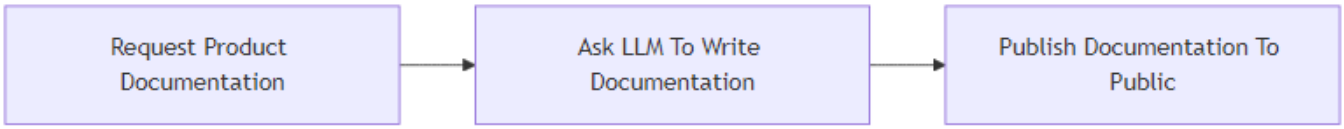
```
// Install the Semantic Kernel Process Framework Local Runtime package
dotnet add package Microsoft.SemanticKernel.Process.LocalRuntime --version 1.46.0-al-
pha
// or
// Install the Semantic Kernel Process Framework Dapr Runtime package
dotnet add package Microsoft.SemanticKernel.Process.Runtime.Dapr --version 1.46.0-
alpha
```

Illustrative Example: Generating Documentation for a New Product

In this example, we will utilize the Semantic Kernel Process Framework to develop an automated process for creating documentation for a new product. This process will start out simple and evolve as we go to cover more realistic scenarios.

We will start by modeling the documentation process with a very basic flow:

1. `GatherProductInfoStep`: Gather information about the product.
2. `GenerateDocumentationStep`: Ask an LLM to generate documentation from the information gathered in step 1.
3. `PublishDocumentationStep`: Publish the documentation.



Now that we understand our processes, let's build it.

Define the process steps

Each step of a Process is defined by a class that inherits from our base step class. For this process we have three steps:

C#

```
using Microsoft.SemanticKernel.ChatCompletion;
using Microsoft.SemanticKernel;

// A process step to gather information about a product
public class GatherProductInfoStep: KernelProcessStep
{
    [KernelFunction]
    public string GatherProductInformation(string productName)
    {
        Console.WriteLine($"{nameof(GatherProductInfoStep)}:\n\tGathering product information for product named {productName}");

        // For example purposes we just return some fictional information.
        return
            """
            Product Description:
            GlowBrew is a revolutionary AI driven coffee machine with industry leading number of LEDs and programmable light shows. The machine is also capable of brewing coffee and has a built in grinder.
```

```
            Product Features:
            1. **Luminous Brew Technology**: Customize your morning ambiance with
programmable LED lights that sync with your brewing process.
            2. **AI Taste Assistant**: Learns your taste preferences over time and
suggests new brew combinations to explore.
            3. **Gourmet Aroma Diffusion**: Built-in aroma diffusers enhance your
coffee's scent profile, energizing your senses before the first sip.

            Troubleshooting:
            - **Issue**: LED Lights Malfunctioning
                - **Solution**: Reset the lighting settings via the app. Ensure the
LED connections inside the GlowBrew are secure. Perform a factory reset if necessary.
            """;
    }
}

// A process step to generate documentation for a product
public class GenerateDocumentationStep :
KernelProcessStep<GeneratedDocumentationState>
{
    private GeneratedDocumentationState _state = new();

    private string systemPrompt =
            """
            Your job is to write high quality and engaging customer facing documenta-
tion for a new product from Contoso. You will be provide with information
            about the product in the form of internal documentation, specs, and trou-
bleshooting guides and you must use this information and
            nothing else to generate the documentation. If suggestions are provided
on the documentation you create, take the suggestions into account and
            rewrite the documentation. Make sure the product sounds amazing.
            """;

    // Called by the process runtime when the step instance is activated. Use this to
load state that may be persisted from previous activations.
    override public ValueTask
ActivateAsync(KernelProcessStepState<GeneratedDocumentationState> state)
    {
        this._state = state.State!;
        this._state.ChatHistory ??= new ChatHistory(systemPrompt);

        return base.ActivateAsync(state);
    }

    [KernelFunction]
    public async Task GenerateDocumentationAsync(Kernel kernel,
KernelProcessStepContext context, string productInfo)
    {
        Console.WriteLine($"[{nameof(GenerateDocumentationStep)}]:\tGenerating docu-
mentation for provided productInfo...");

        // Add the new product info to the chat history
        this._state.ChatHistory!.AddUserMessage($"Product Info:\n{productInfo.Title}
- {productInfo.Content}");
```

```
        // Get a response from the LLM
        IChatCompletionService chatCompletionService =
kernel.GetRequiredService<IChatCompletionService>();
        var generatedDocumentationResponse = await
chatCompletionService.GetChatMessageContentAsync(this._state.ChatHistory!);

        DocumentInfo generatedContent = new()
        {
            Id = Guid.NewGuid().ToString(),
            Title = $"Generated document - {productInfo.Title}",
            Content = generatedDocumentationResponse.Content!,
        };

        this._state!.LastGeneratedDocument = generatedContent;

        await context.EmitEventAsync("DocumentationGenerated", generatedContent);
    }

    public class GeneratedDocumentationState
    {
        public DocumentInfo LastGeneratedDocument { get; set; } = new();
        public ChatHistory? ChatHistory { get; set; }
    }
}

// A process step to publish documentation
public class PublishDocumentationStep : KernelProcessStep
{
    [KernelFunction]
    public DocumentInfo PublishDocumentation(DocumentInfo document)
    {
        // For example purposes we just write the generated docs to the console
        Console.WriteLine($"[{nameof(PublishDocumentationStep)}]:\tPublishing product
documentation approved by user: \n{document.Title}\n{document.Content}");
        return document;
    }
}

// Custom classes must be serializable
public class DocumentInfo
{
    public string Id { get; set; } = string.Empty;
    public string Title { get; set; } = string.Empty;
    public string Content { get; set; } = string.Empty;
}
```

The code above defines the three steps we need for our Process. There are a few points to call out here:

- In Semantic Kernel, a `KernelFunction` defines a block of code that is invocable by native code or by an LLM. In the case of the Process framework, `KernelFunction`s are the invocable members of a Step and each step requires at least one KernelFunction to be defined.
- The Process Framework has support for stateless and stateful steps. Stateful steps automatically checkpoint their progress and maintain state over multiple invocations. The `GenerateDocumentationStep` provides an example of this where the `GeneratedDocumentationState` class is used to persist the `ChatHistory` and `LastGeneratedDocument` object.
- Steps can manually emit events by calling `EmitEventAsync` on the `KernelProcessStepContext` object. To get an instance of `KernelProcessStepContext` just add it as a parameter on your KernelFunction and the framework will automatically inject it.

Define the process flow

C#

```
// Create the process builder
ProcessBuilder processBuilder = new("DocumentationGeneration");

// Add the steps
var infoGatheringStep = processBuilder.AddStepFromType<GatherProductInfoStep>();
var docsGenerationStep = processBuilder.AddStepFromType<GenerateDocumentationStep>();
var docsPublishStep = processBuilder.AddStepFromType<PublishDocumentationStep>();

// Orchestrate the events
processBuilder
    .OnInputEvent("Start")
    .SendEventTo(new(infoGatheringStep));

infoGatheringStep
    .OnFunctionResult()
    .SendEventTo(new(docsGenerationStep));

docsGenerationStep
    .OnFunctionResult()
    .SendEventTo(new(docsPublishStep));
```

There are a few things going on here so let's break it down step by step.

1. Create the builder: Processes use a builder pattern to simplify wiring everything up. The builder provides methods for managing the steps within a process and for managing the lifecycle of the process.

2. Add the steps: Steps are added to the process by calling the `AddStepFromType` method of the builder. This allows the Process Framework to manage the lifecycle of steps by

instantiating instances as needed. In this case we've added three steps to the process and created a variable for each one. These variables give us a handle to the unique instance of each step that we can use next to define the orchestration of events.

3. Orchestrate the events: This is where the routing of events from step to step are defined. In this case we have the following routes:

    - When an external event with `id = Start` is sent to the process, this event and its associated data will be sent to the `infoGatheringStep` step.
    - When the `infoGatheringStep` finishes running, send the returned object to the `docsGenerationStep` step.
    - Finally, when the `docsGenerationStep` finishes running, send the returned object to the `docsPublishStep` step.

> **Tip**
>
> ***Event Routing in Process Framework:*** You may be wondering how events that are sent to steps are routed to KernelFunctions within the step. In the code above, each step has only defined a single KernelFunction and each KernelFunction has only a single parameter (other than Kernel and the step context which are special, more on that later). When the event containing the generated documentation is sent to the `docsPublishStep` it will be passed to the `document` parameter of the `PublishDocumentation` KernelFunction of the `docsGenerationStep` step because there is no other choice. However, steps can have multiple KernelFunctions and KernelFunctions can have multiple parameters, in these advanced scenarios you need to specify the target function and parameter.

Build and run the Process

C#

```
// Configure the kernel with your LLM connection details
Kernel kernel = Kernel.CreateBuilder()
    .AddAzureOpenAIChatCompletion("myDeployment", "myEndpoint", "myApiKey")
    .Build();

// Build and run the process
var process = processBuilder.Build();
await process.StartAsync(kernel, new KernelProcessEvent { Id = "Start", Data =
"Contoso GlowBrew" });
```

We build the process and call `StartAsync` to run it. Our process is expecting an initial external event called `Start` to kick things off and so we provide that as well. Running this process shows the following output in the Console:

```
GatherProductInfoStep: Gathering product information for product named Contoso
GlowBrew
GenerateDocumentationStep: Generating documentation for provided productInfo
PublishDocumentationStep: Publishing product documentation:

# GlowBrew: Your Ultimate Coffee Experience Awaits!

Welcome to the world of GlowBrew, where coffee brewing meets remarkable technology!
At Contoso, we believe that your morning ritual shouldn't just include the perfect
cup of coffee but also a stunning visual experience that invigorates your senses. Our
revolutionary AI-driven coffee machine is designed to transform your kitchen routine
into a delightful ceremony.

## Unleash the Power of GlowBrew

### Key Features

- **Luminous Brew Technology**
  - Elevate your coffee experience with our cutting-edge programmable LED lighting.
GlowBrew allows you to customize your morning ambiance, creating a symphony of colors
that sync seamlessly with your brewing process. Whether you need a vibrant wake-up
call or a soothing glow, you can set the mood for any moment!

- **AI Taste Assistant**
  - Your taste buds deserve the best! With the GlowBrew built-in AI taste assistant,
the machine learns your unique preferences over time and curates personalized brew
suggestions just for you. Expand your coffee horizons and explore delightful new com-
binations that fit your palate perfectly.

- **Gourmet Aroma Diffusion**
  - Awaken your senses even before that first sip! The GlowBrew comes equipped with
gourmet aroma diffusers that enhance the scent profile of your coffee, diffusing rich
aromas that fill your kitchen with the warm, inviting essence of freshly-brewed
bliss.

### Not Just Coffee - An Experience

With GlowBrew, it's more than just making coffee-it's about creating an experience
that invigorates the mind and pleases the senses. The glow of the lights, the aroma
wafting through your space, and the exceptional taste meld into a delightful ritual
that prepares you for whatever lies ahead.

## Troubleshooting Made Easy

While GlowBrew is designed to provide a seamless experience, we understand that tech-
nology can sometimes be tricky. If you encounter issues with the LED lights, we've
got you covered:
```

```
- **LED Lights Malfunctioning?**
  - If your LED lights aren't working as expected, don't worry! Follow these steps to
restore the glow:
    1. **Reset the Lighting Settings**: Use the GlowBrew app to reset the lighting
settings.
    2. **Check Connections**: Ensure that the LED connections inside the GlowBrew are
secure.
    3. **Factory Reset**: If you're still facing issues, perform a factory reset to
rejuvenate your machine.

With GlowBrew, you not only brew the perfect coffee but do so with an ambiance that
excites the senses. Your mornings will never be the same!

## Embrace the Future of Coffee

Join the growing community of GlowBrew enthusiasts today, and redefine how you expe-
rience coffee. With stunning visual effects, customized brewing suggestions, and aro-
matic enhancements, it's time to indulge in the delightful world of GlowBrew-where
every cup is an adventure!

### Conclusion

Ready to embark on an extraordinary coffee journey? Discover the perfect blend of
technology and flavor with Contoso's GlowBrew. Your coffee awaits!
```

What's Next?

Our first draft of the documentation generation process is working but it leaves a lot to be desired. At a minimum, a production version would need:

- A proof reader agent that will grade the generated documentation and verify that it meets our standards of quality and accuracy.
- An approval process where the documentation is only published after a human approves it (human-in-the-loop).

Add a proof reader agent to our process...

Last updated on 02/26/2025